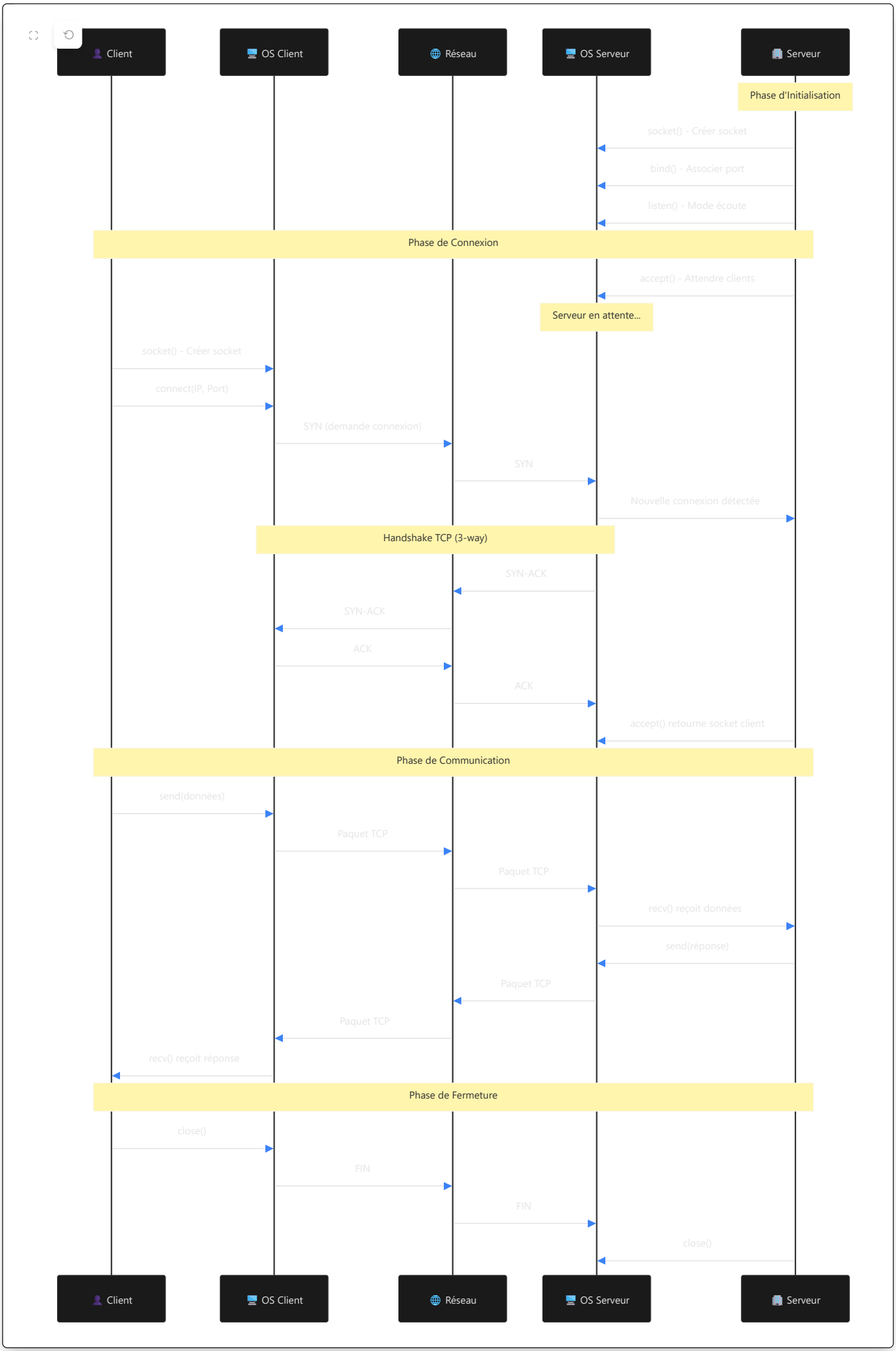
R3.05 - TD Socket

☰ TOC

Introduction : fondamentaux

Un **socket** est un point d'extrémité de communication bidirectionnelle entre deux processus, qu'ils s'exécutent sur la même machine ou sur des machines distantes. Cette abstraction révolutionnaire unifie la communication locale et réseau sous une API cohérente, permettant aux développeurs de traiter indifféremment les connexions locales et distantes.

Les sockets implémentent naturellement le paradigme **client-serveur**, modèle architectural fondamental de la communication réseau moderne. Cette approche asymétrique définit des rôles clairs et des responsabilités distinctes.


Client
OS Client
Réseau
OS Serveur
Serveur
Phase d'Initialisation
socket() - Créer socket
bind() - Associer port
listen() - Mode écoute
Phase de Connexion
accept() - Attendre clients
Serveur en attente...
socket() - Créer socket
connect(IP, Port)
SYN (demande connexion)
SYN
Nouvelle connexion détectée
Handshake TCP (3-way)
SYN-ACK
SYN-ACK
ACK
ACK
accept() retourne socket client
Phase de Communication
send(données)
Paquet TCP
Paquet TCP
recv() reçoit données
send(réponse)
Paquet TCP
Paquet TCP
recv() reçoit réponse
Phase de Fermeture
close()
FIN
FIN
close()
Client
OS Client
Réseau
OS Serveur
Serveur

Types de sockets et domaines de communication

Domaines de communication :

- **AF_INET (Internet Protocol v4)** : Communication via réseau IP standard, adressage par IP + Port, support mondial
- **AF_INET6 (Internet Protocol v6)** : Évolution d'IPv4 avec adresses 128 bits et fonctionnalités avancées
- **AF_UNIX/AF_LOCAL (Unix Domain Sockets)** : Communication locale uniquement, performance optimale, sécurité renforcée

Protocoles de transport :

- **SOCK_STREAM (TCP)** : Connexion orientée, fiabilité garantie, ordre préservé, contrôle de flux
- **SOCK_DGRAM (UDP)** : Sans connexion, rapide, non-fiable, idéal pour applications temps-réel

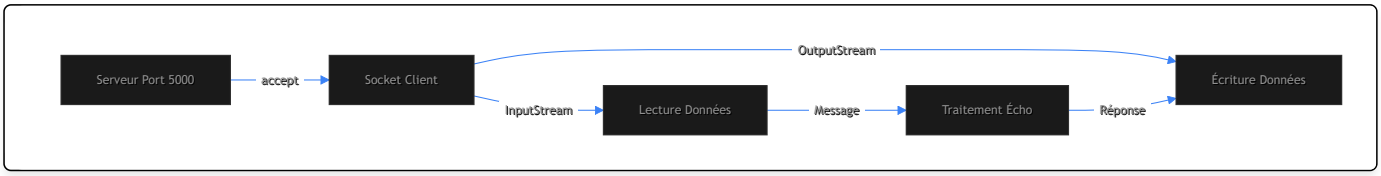
> En Java, les mécanismes sont simplifiés et cachés, et nous allons les manipuler ci-dessous. N'hésitez pas à consulter le guide officiel Oracle sur les sockets Java : Socket Programming

Étape 1 : serveur écho basique

Objectif technique

Implémenter un serveur TCP capable d'accepter une connexion unique, de recevoir des messages textuels et de les renvoyer tels quels (écho). Cette étape introduit les concepts fondamentaux : création de socket serveur, acceptation de connexion, flux d'entrée/sortie et cycle de vie d'une connexion.

Architecture du système



Code de départ

```java
import java.net.*;
import java.io.*;

public class Serveur {
    public static void main(String[] args) throws Exception {
        // 1. Création du serveur et liaison au port 5000

        while(true) {
            // 2. Acceptation d'une connexion cliente
            // 3. Création des canaux input/output
            // 4. Boucle d'écho (lecture/écriture)
            // 5. Fermeture de la connexion client
        }
    }
}
```

Création de la socket serveur

La première étape consiste à créer une **socket serveur**. Le principe repose sur un mécanisme IPC (Inter-Process Communication) à travers un **descripteur de fichier**. En gros, cela permet de manipuler les connexions réseau comme s'il s'agissait d'un fichier normal. Quand on crée une socket serveur, cela **expose** ou **ouvre** un port de communication sur la machine.

```java
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Serveur démarré sur le port 5000");
```

Explication technique :

- `ServerSocket` encapsule l'appel système `socket()` + `bind()` + `listen()` en une seule opération
- Le port 5000 devient le point d'écoute TCP sur **toutes les interfaces réseau** de la machine (0.0.0.0)
- Le système d'exploitation alloue un descripteur de fichier et crée une file d'attente pour les connexions entrantes (backlog par défaut : 50 connexions)

> On peut par mesure de sécurité restreindre les interfaces réseau qui y ont accès, mais ce n'est pas le but ici. En principe, si vous connaissez l'IP de la machine du voisin, vous devriez être capable de communiquer avec elle.

Vérification système : Une fois le serveur lancé, vous pouvez observer le port ouvert avec :

```
netstat -an | grep 5000
# Résultat attendu : tcp  0  0  0.0.0.0:5000  0.0.0.0:*  LISTEN
```

Acceptation d'une connexion cliente

Pour l'instant, vous avez juste dit que l'on pouvait ouvrir une connexion avec le serveur. L'étape suivante consiste à simplement **accepter** une connexion. Dans la boucle `while`, ajoutez :

```
Socket client = serverSocket.accept();
System.out.println("Client connecté : " + client.getInetAddress());
```

Explication technique :

- `accept()` est un **appel bloquant** : le thread s'endort jusqu'à ce qu'une connexion TCP soit établie
- Au niveau système, cela correspond à l'extraction d'une connexion de la file d'attente créée par `listen()`
- Le retour est une nouvelle `Socket` représentant le canal bidirectionnel avec ce client spécifique
- Cette `Socket` est un **File Descriptor (FD)** au même titre que la socket serveur, mais elle représente une connexion établie (état ESTABLISHED du TCP)

Différence conceptuelle importante :

- `ServerSocket` : socket d'écoute passive, ne transporte jamais de données applicatives
- `Socket` (retournée par accept) : socket de communication active, utilisée pour les échanges de données

Création des flux de communication

Il faut maintenant passer par des objets qui représentent les **flux de données**, que l'on a vus rapidement dans le TP sur les processus : `client.getInputStream()` et `client.getOutputStream()`.

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(client.getInputStream())
);
PrintWriter out = new PrintWriter(client.getOutputStream(), true);
```

Explication technique :

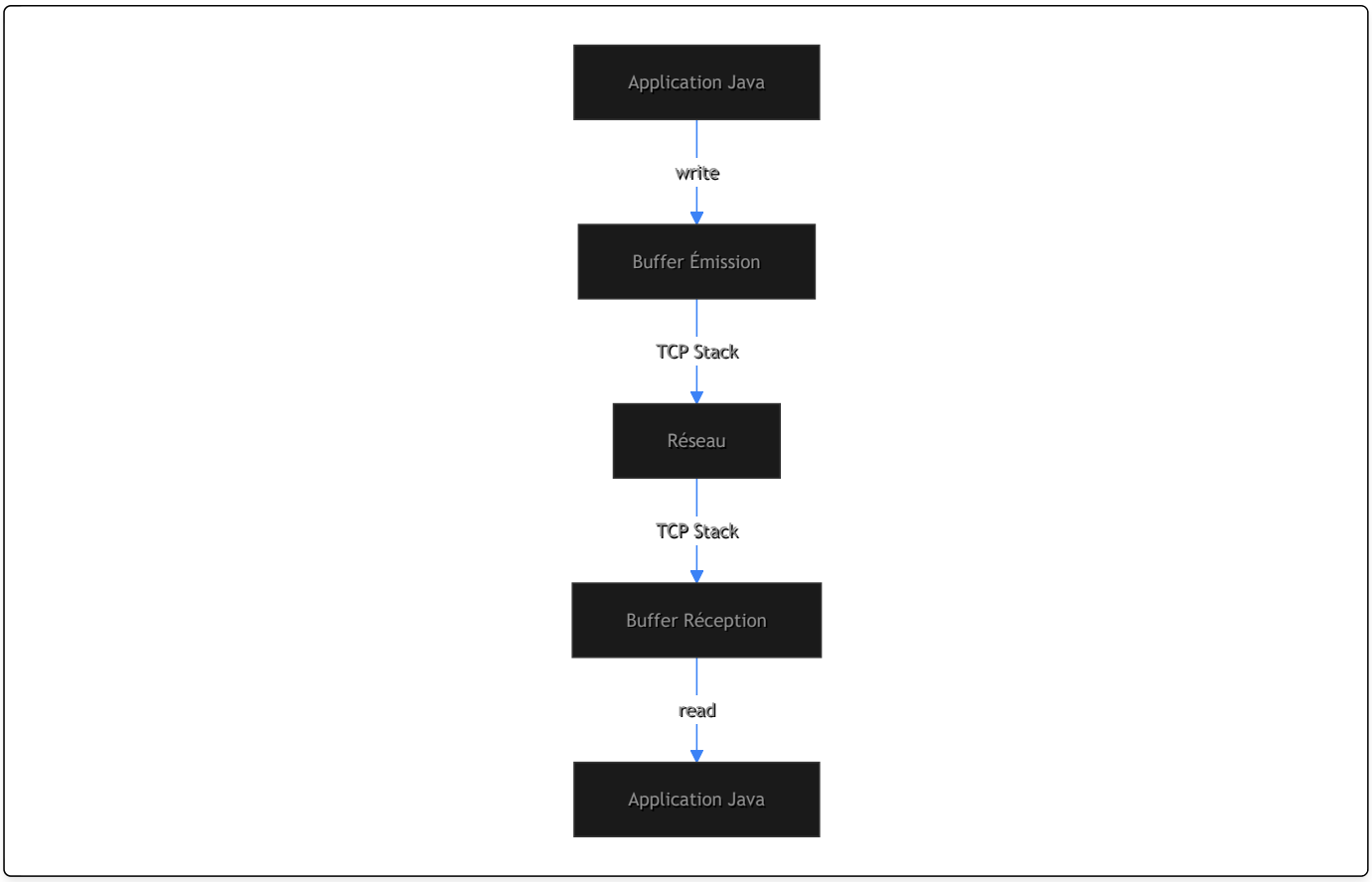
`client.getInputStream()` :

- Retourne un `InputStream` brut, flux d'octets bas niveau lisant depuis le buffer de réception TCP du kernel
- Les données arrivent du réseau via le socket et sont bufferisées par le système d'exploitation
- `InputStreamReader` convertit ce flux d'octets en flux de caractères (décodage UTF-8 par défaut)
- `BufferedReader` ajoute une couche de bufferisation applicative et permet la lecture ligne par ligne avec `readLine()`

`client.getOutputStream()` :

- Retourne un `OutputStream` brut, flux d'octets bas niveau écrivant dans le buffer d'émission TCP du kernel
- `PrintWriter` encapsule ce flux pour faciliter l'écriture de texte formaté
- Le paramètre `true` active l'**auto-flush** : chaque `println()` vide immédiatement le buffer vers le réseau (appel système `write()`)

Rappel architecture TCP :



Boucle de traitement écho

```
String ligne;
while ((ligne = in.readLine()) != null) {
    System.out.println("Reçu : " + ligne);
    out.println("ECHO: " + ligne);
}
```

Explication technique :

- `readLine()` bloque jusqu'à recevoir un caractère de fin de ligne (`\n` ou `\r\n`)
- La condition `!= null` détecte la **fermeture du flux** côté client (réception d'un paquet FIN TCP)
- Chaque message reçu est immédiatement renvoyé avec le préfixe "ECHO:"
- Le flush automatique garantit l'envoi immédiat sans attendre un buffer plein

Comportement TCP sous-jacent :

- Les données sont transmises en segments TCP (taille variable selon la MTU réseau)
- L'algorithme de Nagle peut regrouper plusieurs `println()` en un seul paquet si le flush est désactivé
- Le contrôle de flux TCP ajuste automatiquement le débit selon la congestion réseau

Fermeture propre des ressources

```
System.out.println("Client déconnecté");
in.close();
out.close();
client.close();
```

Explication technique :

- `in.close()` et `out.close()` ferment les flux applicatifs
- `client.close()` envoie un paquet **FIN** TCP pour initier la fermeture gracieuse (four-way handshake)
- Le descripteur de fichier est libéré, le port local redevient disponible
- Sans fermeture explicite, le socket reste en état `TIME_WAIT` pendant 2×MSL (généralement 60 secondes)

Implémentation du client

Maintenant que le serveur est prêt à accepter des connexions, nous allons créer un client capable de s'y connecter et d'échanger des messages.

Code de départ

```
import java.net.*;
import java.io.*;
```

```
public class Client {
    public static void main(String[] args) throws Exception {
        // 1. Connexion au serveur
        // 2. Flux réseau (vers/depuis le serveur)
        // 3. Flux clavier (entrée utilisateur)
        // 4. Boucle d'envoi/réception
        // 5. Fermeture
    }
}
```

Connexion au serveur

```
Socket socket = new Socket("localhost", 5000);
System.out.println("Connecté au serveur");
```

Explication technique :

- `new Socket("localhost", 5000)` initie le **three-way handshake** TCP (SYN → SYN-ACK → ACK)
- "localhost" est résolu en 127.0.0.1 (interface loopback)
- Le système d'exploitation alloue automatiquement un **port éphémère** (généralement 49152-65535)
- La connexion est établie quand le constructeur retourne (ou lève `ConnectException` si refusée)

Création des flux de communication

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream())
);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader clavier = new BufferedReader(
    new InputStreamReader(System.in)
);
```

Explication technique :

- `socket.getInputStream()` : lit les données envoyées par le serveur
- `socket.getOutputStream()` : écrit les données à envoyer au serveur
- `System.in` : lit les entrées utilisateur depuis le clavier (flux standard)

Boucle d'envoi et réception

```
String texte;
while ((texte = clavier.readLine()) != null) {
    out.println(texte);
    String reponse = in.readLine();
    System.out.println(reponse);
}
socket.close();
```

Explication technique :

- `clavier.readLine()` bloque jusqu'à ce que l'utilisateur tape Entrée
- `out.println(texte)` envoie le message au serveur (avec flush automatique)
- `in.readLine()` bloque jusqu'à recevoir la réponse du serveur
- Ctrl+D (Linux/Mac) ou Ctrl+Z (Windows) termine la boucle

Test de la communication

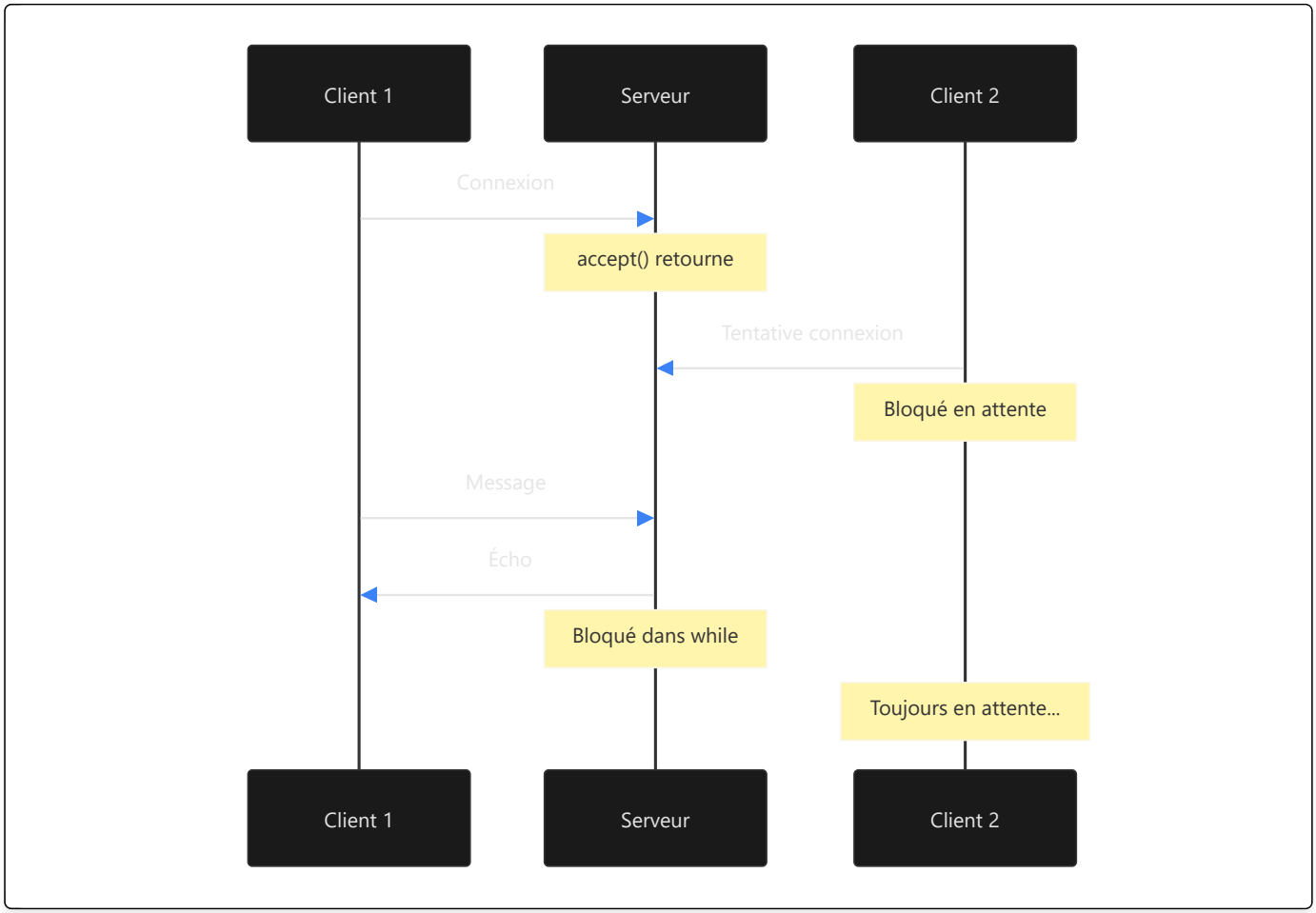
Terminal 1 (serveur) :

```
javac Serveur.java
java Serveur
# Affiche : Serveur démarré sur le port 5000
```

Terminal 2 (client) :

```
javac Client.java
java Client
# Affiche : Connecté au serveur
Bonjour
# Affiche : ECHO: Bonjour
```

Problème majeur de cette implémentation :



Le serveur ne peut traiter qu'**un seul client à la fois**. Les autres connexions restent dans la file d'attente du système (backlog), créant une latence inacceptable pour les applications réelles.

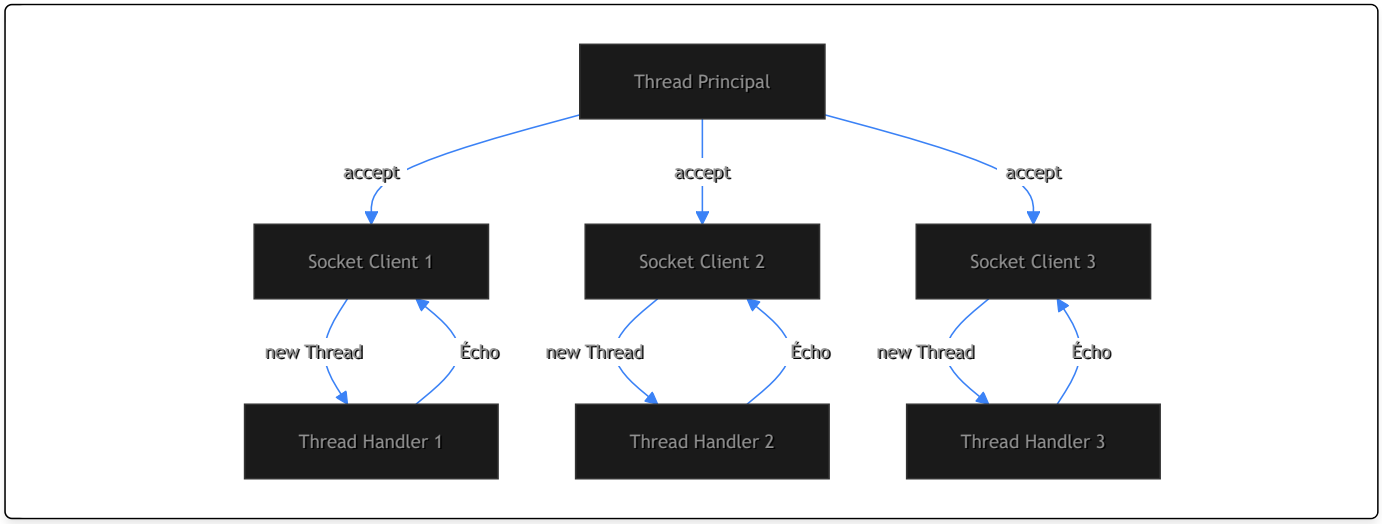
Solution : introduire le multi-threading (Étape 2).

Étape 2 : serveur multi-clients

Objectif technique

Transformer le serveur monothread en serveur concurrent capable de gérer **plusieurs clients simultanément** via l'allocation d'un thread dédié par connexion. Cette approche introduit les concepts de concurrence, isolation des contextes d'exécution et gestion du cycle de vie des threads.

Architecture concurrente



Principe du modèle thread-per-connection

Séparation des responsabilités :

- **Thread principal** : boucle d'acceptation dédiée, création rapide de threads handlers
- **Threads workers** : un par client, exécutent la logique métier (lecture/écho) de manière isolée

Avantages :

- Scalabilité immédiate (N clients = N threads)
- Isolation des erreurs (un crash client n'affecte pas les autres)
- Simplicité de développement (code séquentiel par client)

Inconvénients (à considérer pour la production) :

- Coût mémoire élevé (chaque thread = ~1 Mo de stack)
- Overhead de context switching au-delà de quelques milliers de threads
- Limite système du nombre de threads (`ulimit -u`)

Implémentation avec Runnable

```
import java.net.*;
import java.io.*;

public class ServeurMulti {
    public static void main(String[] args) throws Exception {
        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("Serveur multi-clients démarré");

        while (true) {
            Socket client = serverSocket.accept();
            System.out.println("Nouveau client : " + client.getInetAddress());

            // Délégation à un thread dédié
            Thread t = new Thread(new ClientHandler(client));
            t.start();
        }
    }
}

class ClientHandler implements Runnable {
    private Socket client;

    public ClientHandler(Socket socket) {
        this.client = socket;
    }

    @Override
    public void run() {
         // TODO
    }
}
```

Explication technique approfondie

Création du thread :

```
Thread t = new Thread(new ClientHandler(client));
t.start();
```

- `new Thread(Runnable)` crée un nouveau contexte d'exécution mais ne le démarre pas
- `start()` invoque l'appel système natif (pthread_create sur Linux) qui :
- Alloue une stack dédiée (taille configurable via -Xss, défaut 1 Mo)
- Place le thread dans l'état RUNNABLE du scheduler OS
- Planifie l'exécution de la méthode `run()` dès qu'un cœur CPU se libère

Passage du socket au thread :

- Le constructeur `ClientHandler(Socket)` **transfère la propriété** du socket au thread
- Le thread principal ne doit **jamais** manipuler ce socket après le démarrage
- Cette isolation garantit qu'aucune race condition ne survient sur les flux I/O

Bloc try-finally :

```
finally {
    try { client.close(); } catch (IOException ignored) {}
}
```

- Garantit la fermeture du socket même en cas d'exception (IOException, RuntimeException)
- Le double try-catch gère le cas rare où `close()` lui-même lève une exception

- Sans ce bloc, un client malveillant pourrait épuiser les descripteurs de fichiers disponibles

Identification des threads :

```
Thread.currentThread().getName()
```

- Chaque thread a un nom auto-généré (Thread-0, Thread-1, etc.)
- Utile pour le debugging et la corrélation des logs
- Peut être personnalisé avec `t.setName("Client-" + client.getPort())`

Test avec plusieurs clients

Pour observer le comportement concurrent, lancez plusieurs clients simultanément :

Terminal 1 (serveur) :

```
java ServeurMulti
```

Terminaux 2, 3, 4 (clients) :

```
java Client
```

Observation attendue :

```
[Thread-0] Reçu : Hello from client 1
[Thread-1] Reçu : Hello from client 2
[Thread-2] Reçu : Hello from client 3
```

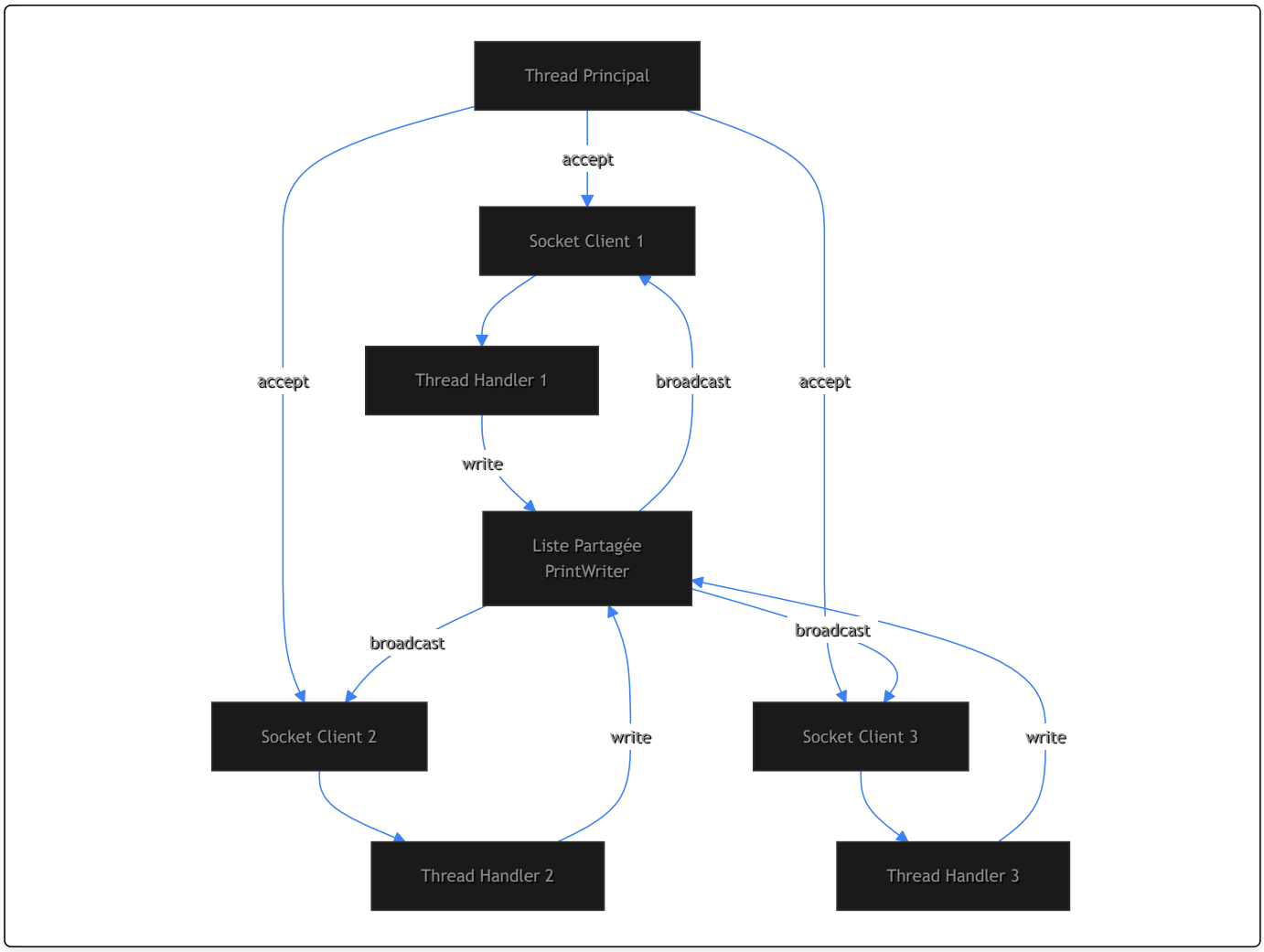
Les messages sont entrelacés, démontrant l'exécution parallèle réelle.

Étape 3 : broadcast - diffusion à tous les clients

Objectif technique

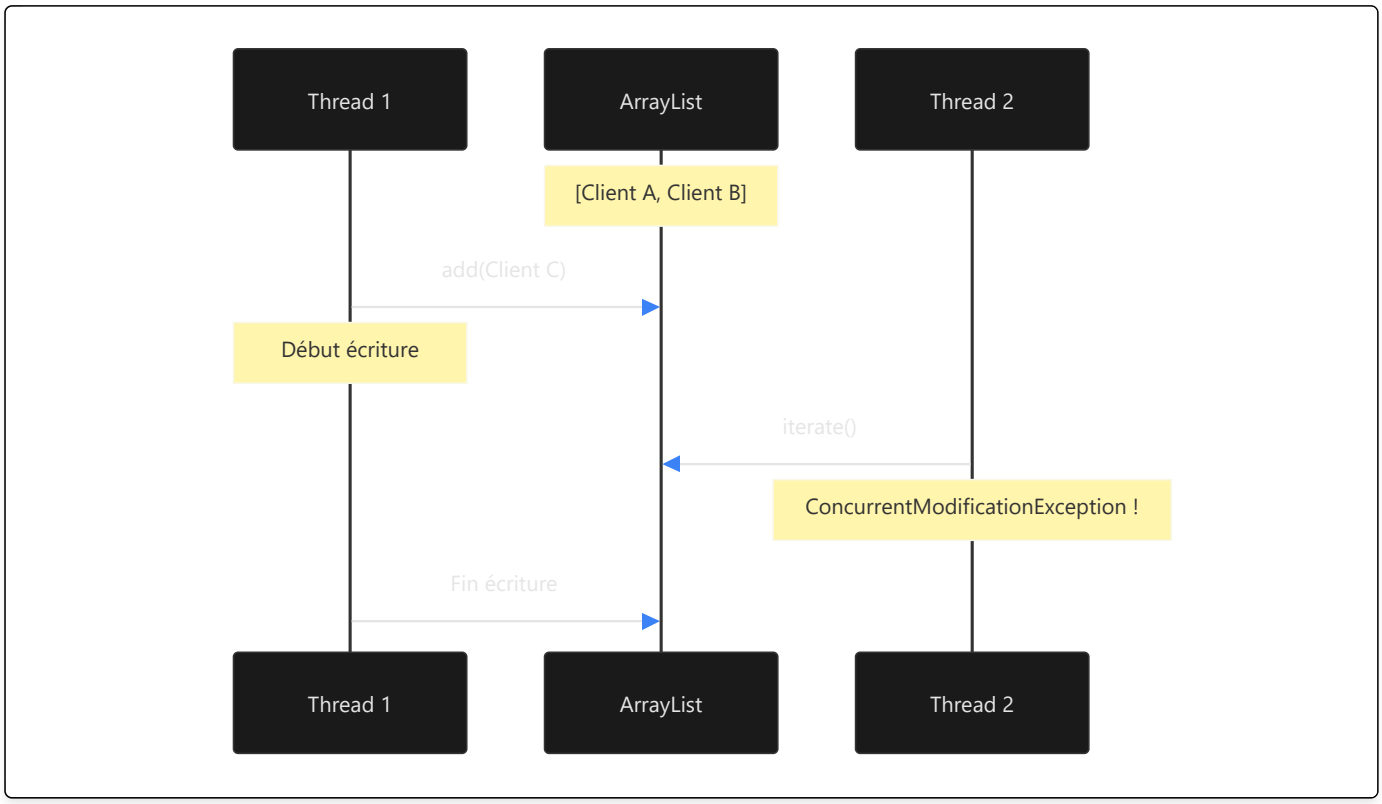
Implémenter un mécanisme de **diffusion (broadcast)** où chaque message reçu par un client est retransmis à **tous** les clients connectés. Cela introduit les concepts de **synchronisation inter-threads**, de **structures partagées** et de **gestion de la concurrence** via les mécanismes Java (synchronized, Collections thread-safe).

Architecture du système de broadcast



Problématique de synchronisation

Scénario de race condition sans synchronisation :



Problèmes sans synchronisation :

1. **Modification concurrente** : ajout/suppression pendant itération → `ConcurrentModificationException`
2. **Visibilité mémoire** : un thread peut ne pas voir les modifications d'un autre (cache CPU)
3. **Atomicité** : opérations composées (vérifier + ajouter) peuvent être entrelacées

Implémentation avec synchronisation

```
import java.net.*;
import java.io.*;
import java.util.*;

public class ServeurChat {
    // Liste thread-safe des flux de sortie
    private static final List<PrintWriter> clients =
        Collections.synchronizedList(new ArrayList<>());

    public static void main(String[] args) throws Exception {
        ServerSocket serverSocket = new ServerSocket(5000);
        System.out.println("Serveur Chat démarré");

        while (true) {
            Socket client = serverSocket.accept();
            PrintWriter out = new PrintWriter(client.getOutputStream(), true);
            // TODO
            System.out.println("Client connecté. Total : " + clients.size());
            new Thread(new ChatHandler(client, out)).start();
        }
    }

    static class ChatHandler implements Runnable {
        private Socket socket;
        private PrintWriter out;

        public ChatHandler(Socket socket, PrintWriter out) {
            this.socket = socket;
            this.out = out;
        }

        @Override
        public void run() {
        }

        private void broadcast(String message) {
            // TODO
        }
    }
}
```

Explication technique détaillée

Collections thread-safe :

```
Collections.synchronizedList(new ArrayList<>())
```

Ce que fait cette ligne :

- Encapsule une `ArrayList` normale dans un wrapper synchronisé
- Chaque méthode (`add` , `remove` , `get`) est protégée par un verrou intrinsèque
- Mais **attention** : l'itération n'est pas automatiquement thread-safe !

Pourquoi l'itération nécessite une synchronisation manuelle :

```
// DANGEREUX - ConcurrentModificationException possible
for (PrintWriter w : clients) { w.println(msg); }

// CORRECT - Verrou explicite
synchronized (clients) {
    for (PrintWriter w : clients) { w.println(msg); }
}
```

L'itération fait plusieurs appels internes (`hasNext()` , `next()`), qui doivent être atomiques.

Bloc synchronized

```
synchronized (clients) {
    clients.add(out);
}
```

Mécanisme bas niveau :

- Chaque objet Java possède un **monitor** (verrou intrinsèque)
- `synchronized (obj)` acquiert ce monitor pour le thread courant
- Si un autre thread le détient, le thread actuel est bloqué (état BLOCKED)
- À la sortie du bloc, le monitor est libéré automatiquement (même en cas d'exception)

Équivalent en pseudo-code système :

```
pthread_mutex_lock(&clients_mutex);
clients.add(out);
pthread_mutex_unlock(&clients_mutex);
```

Diffusion des messages

```
private void broadcast(String message) {
    synchronized (clients) {
        for (PrintWriter writer : clients) {
            writer.println(">>> " + message);
        }
    }
}
```

Analyse de performance :

- Le verrou est maintenu pendant **toute la durée** de l'envoi à tous les clients
- Si un client a une connexion lente, **tous les autres threads sont bloqués**
- Dans un système de production, on préférerait :
- Copier la liste sous verrou : `List<PrintWriter> snapshot = new ArrayList<>(clients);`
- Itérer sur la copie sans verrou
- Gérer les erreurs d'envoi individuellement

Gestion de la déconnexion

```
finally {
    synchronized (clients) {
        clients.remove(out);
    }
    try { socket.close(); } catch (IOException ignored) {}
}
```

Ordre critique des opérations :

1. Retirer le client de la liste (sinon messages envoyés à un socket fermé)
2. Fermer le socket (libère les ressources système)

Sans le finally :

- Si une exception survient, le `PrintWriter` reste dans la liste
- Les futurs broadcasts tentent d'écrire sur un socket fermé → IOException silencieuse
- Fuite de mémoire progressive (la liste grossit indéfiniment)

Variante : broadcast sauf à l'émetteur

Dans de nombreux systèmes de chat, on veut éviter que l'utilisateur reçoive son propre message en écho. Pour cela, on peut modifier la méthode `broadcast` pour exclure l'émetteur :

```
private void broadcastExcept(String message) {
    synchronized (clients) {
        for (PrintWriter writer : clients) {
            if (writer != out) {  // Exclut l'émetteur
                writer.println(">>> " + message);
            }
        }
    }
}
```

Explication technique :

- La comparaison `writer != out` utilise l'**identité d'objet** (pas `equals()`)
- Chaque `PrintWriter` est unique par connexion, donc cette comparaison identifie précisément l'émetteur
- Cette technique évite la redondance : l'utilisateur voit son message localement sans le recevoir du serveur

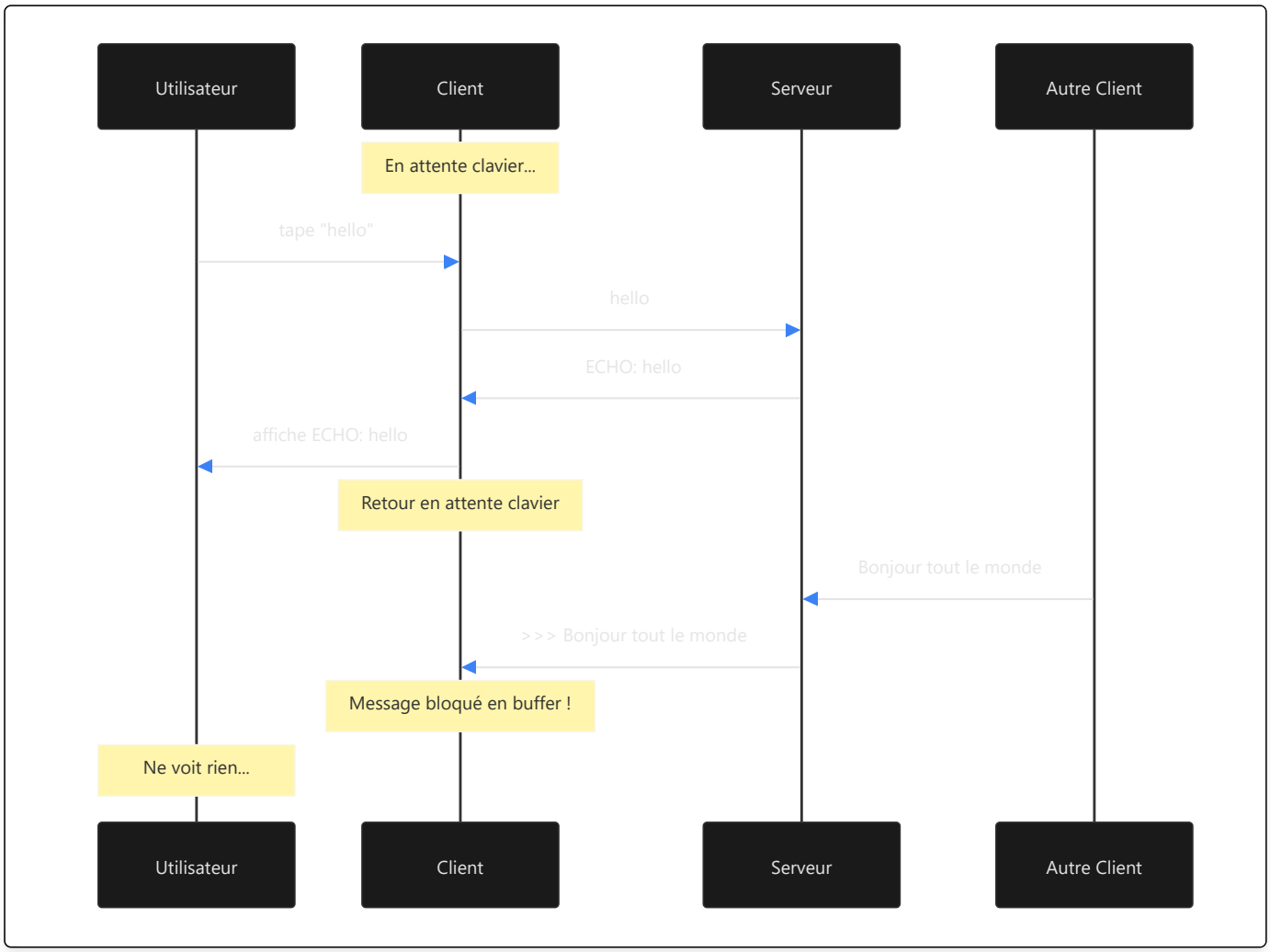
Utilisation :

```
while ((ligne = in.readLine()) != null) {
    System.out.println("Message reçu : " + ligne);
    broadcastExcept(ligne);  // Diffuse à tous sauf l'émetteur
}
```

Étape 4 : client asynchrone

Problématique : blocage du client

Le client actuel présente un défaut architectural majeur dans le contexte d'un chat multi-utilisateurs :



Explication technique du problème :

```
while ((texte = clavier.readLine()) != null) {  // BLOQUE ICI
    out.println(texte);
    String reponse = in.readLine();  // ET ICI
    System.out.println(reponse);
}
```

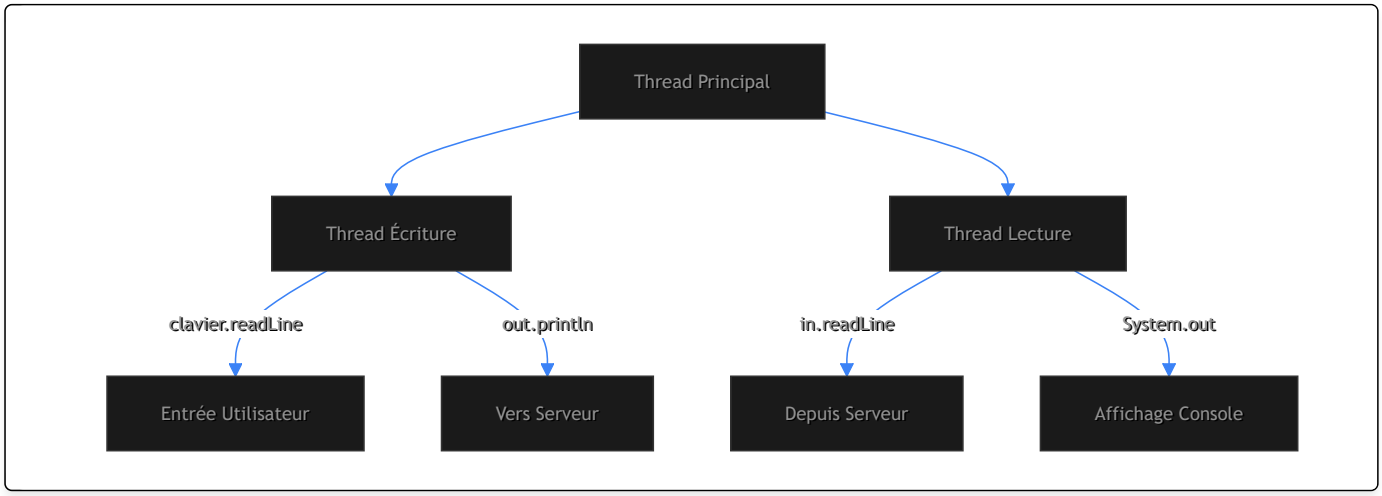
- `clavier.readLine()` est un **appel bloquant** : le thread attend indéfiniment une entrée utilisateur
- Pendant ce blocage, les messages broadcast du serveur arrivent dans le buffer TCP mais ne sont **jamais lus**
- Le client ne peut pas **simultanément** attendre une entrée clavier ET lire les messages réseau

Conséquence pour l'utilisateur :

- Les messages des autres utilisateurs n'apparaissent que lorsqu'il envoie lui-même un message
- L'expérience utilisateur est catastrophique : le chat semble "cassé"

Solution : architecture bi-thread

Pour résoudre ce problème, nous devons séparer les responsabilités en **deux threads indépendants** :



Thread 1 (Écriture) : lit le clavier et envoie au serveur
Thread 2 (Lecture) : lit le serveur et affiche en continu

Implémentation du client asynchrone

```
import java.net.*;
import java.io.*;

public class ClientAsync {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 5000);
        System.out.println("Connecté au serveur");

        // Thread de lecture (messages du serveur)
        Thread lectureThread = new Thread(new ReceptionHandler(socket));
        lectureThread.start();

        // Thread principal = écriture (envoi au serveur)
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader clavier = new BufferedReader(
            new InputStreamReader(System.in)
        );

        String texte;
        while ((texte = clavier.readLine()) != null) {
            out.println(texte);
        }

        socket.close();
    }
}

class ReceptionHandler implements Runnable {
    private Socket socket;

    public ReceptionHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
            // TODO
    }
}
```

Explication technique détaillée

Séparation des flux :

```
Thread lectureThread = new Thread(new ReceptionHandler(socket));
lectureThread.start();
```

- Le thread principal conserve la responsabilité de l'**envoi** (écriture)
- Un thread dédié gère la **réception** (lecture)
- Les deux threads partagent le même `Socket`, mais utilisent des flux distincts (`InputStream` vs `OutputStream`)

Sécurité de cette approche :

- TCP garantit que les flux d'entrée et de sortie sont **indépendants** au niveau protocole
- Java garantit que `getInputStream()` et `getOutputStream()` retournent des objets thread-safe
- Aucune synchronisation nécessaire car il n'y a **pas de ressource partagée** modifiable

Cycle de vie des threads :

```
while ((ligne = in.readLine()) != null) {
    System.out.println(ligne);
}
```

- Le thread de lecture boucle indéfiniment jusqu'à ce que le serveur ferme la connexion
- Quand le serveur envoie FIN, `readLine()` retourne `null` et le thread se termine proprement
- Le thread principal termine quand l'utilisateur ferme l'entrée standard (Ctrl+D)

Explication des améliorations :

`setDaemon(true)` :

- Marque le thread de lecture comme **daemon**
- Permet à la JVM de terminer même si ce thread tourne encore
- Évite que l'application reste bloquée après la fermeture du socket

`volatile boolean running` :

- Le mot-clé `volatile` garantit la **visibilité** entre threads
- Quand le thread principal modifie `running`, le thread de lecture voit immédiatement le changement
- Sans `volatile`, le compilateur pourrait cacher la variable dans un registre CPU

Test du client asynchrone

Terminal 1 (serveur) :

```
java ServeurChat
```

Terminal 2 (client A) :

```
java ClientAsync
Bonjour tout le monde
```

Terminal 3 (client B) :

```
java ClientAsync
# Voit immédiatement : >>> Bonjour tout le monde
Salut !
```

Terminal 2 (client A) :

```
# Voit immédiatement : >>> Salut !
```

Les messages apparaissent **instantanément** sans attendre une saisie clavier.

© Jehan-Antoine Vayssade 2011-2025